TPPA PAYMENTS

IT and set-up guide

The server, the first super admin, installing and registering teller PCs and kiosks, resets, data, backups and security.

For: IT

Version 1.0 · 2026-11-10
The screenshots come from a demonstration system: every name, account, amount and code in them is made up.

Contents

1 About this guide

2 The parts and where they run

- What is built and what is still manual

3 The server

- Before you start
- Set it up
- Check it

4 The first super admin

5 Installing the teller app on a counter PC

- What the counter needs
- Install the app
- Create the terminal in the console
- Register the PC
- Choose and calibrate the printer

6 Installing a kiosk

- The hardware and the PC
- The hardware profile and the drivers
- Create and register the kiosk

7 Machine serial and BIR permit

8 Resetting, replacing and setting up again

- Which action to use
- Reset registration
- A terminal the server does not recognise

9 Data on the terminal

10 Backups and updates as they stand

- Backups (manual today)
- Updating the server (manual today)
- Updating the teller and kiosk apps

11 Security practices

12 Before go-live: the checklist

1. About this guide

This guide is for IT. It covers:

- the parts of the system and where each one runs;
- the server, and creating the first super admin;
- installing a teller PC with its Epson LX-310 and registering it;
- installing and registering a kiosk;
- the machine serial and the BIR permit;
- resetting a terminal, setting it up again, and a terminal the server does not recognise;
- the data kept on a terminal;
- backups and updates as they stand today;
- security practices and a checklist before going live.

The commands for the server are in full in `docs/guides/running.md`, part 2, in the code repository. This guide gives the order of the work and what to check; it never contains a password or a secret. Keep those in your password manager.

The screenshots come from a demonstration system: every name, code and address in them is made up. The printer list in them shows "TPPA folder printer", a development stand-in; on a real counter you choose the Epson LX-310.

2. The parts and where they run

Part	What it is	Where it runs
nginx	The only entry from the internet: HTTPS with a Let's Encrypt certificate, HTTP redirected to HTTPS. Sends `/api/...` to the API and everything else to the console.	The server, Docker
API	Sign-in and every business rule; checks each record the terminals send; makes reports and collection files; sends alert emails.	The server, Docker (private network only)
Console	The head-office web console.	The server, Docker (private network only)
PostgreSQL 17	The database, in the `pgdata` Docker volume.	The server, Docker (no published port)
migrate, **certbot**	One-off database migrations before the API starts; certificate renewal.	The server, Docker
Teller app	The counter app, offline-first, with an encrypted local database.	A Windows PC at each counter
Kiosk app	The self-service kiosk app, offline-first, with an encrypted local database.	The kiosk's Windows PC

```
 Internet --> nginx :80 / :443 --+--> /api/*  --> API :4000 --> PostgreSQL :5432
             (Let's Encrypt)    +--> /*      --> console :3000
 Tailscale --> SSH (no public SSH)

 Teller PCs and kiosks --> https://<console address> (the same address as the console)
```

The browser and the terminals use **one address**: the console's (for example `https://console.tppa.example`). On the server, the API, the console and the database listen only on the Docker stack's private network.

There are two servers, set up the same way: **staging** (synthetic test data only, the `staging` branch) and **production** (real data, the `main` branch). Each has its own secrets and its own database; they never share them.

What is built and what is still manual

As of this guide:

- **Built:** the server stack (Docker Compose, nginx, Let's Encrypt, migrations before the API starts), the console, the teller app, the kiosk app with simulated devices, sync, alert emails, resetting a terminal's registration and setting up a terminal the server does not recognise.
- **Manual today:** backups (a nightly database dump you schedule and copy off the server) and updates of the server (by hand, after a backup). Continuous backups, monitoring and an automatic deploy pipeline are not built yet.
- **Not in the repository yet:** an installer or automatic updater for the teller and kiosk apps, and the real drivers for the kiosk's note acceptor, coin acceptor and door sensors (they arrive with the chosen hardware).

3. The server

Before you start

- A server with **Ubuntu 24.04 LTS** and a public IP (production: 4 vCPU, 8 GB, 160 GB SSD, in Singapore; staging: 2 vCPU, 4 GB).
- A DNS `A` record for the console address, pointing to the server.
- A **Tailscale** account: SSH to the server goes only over Tailscale.
- A read-only **deploy key** for the private code repository.

Set it up

Follow `docs/guides/running.md`, sections 2.2 to 2.6. In short:

1. **Prepare the server:** run `deploy/scripts/setup-server.sh` (best from the provider's web console), `sudo tailscale up`, then the script again. It installs Docker and Tailscale, turns on automatic security updates, opens only ports 80 and 443 to the public, closes public SSH, and creates the `tppa` user and `/opt/tppa-pos`.
2. **Get the code** as the `tppa` user into `/opt/tppa-pos/app`, branch `staging` or `main`.
3. **Create the settings file:** copy `deploy/.env.example` to `deploy/.env`, mode 600. Set the console address (`DOMAIN`), the Let's Encrypt email, and a new database password and `BETTER_AUTH_SECRET` for this server (the guide shows how to generate them). Store them in your password manager.
4. **Mail server for alert emails** (optional now, needed before go-live): `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM` in `deploy/.env`. Put the password in single quotes: Docker Compose would otherwise read a `$` in it as a variable.
5. **Start the stack and get the certificate:** `deploy/scripts/tls-init.sh`. It builds the images, starts the database, applies the migrations, starts the API, the console and nginx, then gets the certificate. Renewal is automatic.

The stack sets the API's server settings itself: `NODE_ENV=production`, the console address as the only trusted origin, and two-step sign-in required for **super admins and admins** (`TWO_FACTOR_REQUIRED_ROLES=SUPER_ADMIN,ADMIN`). Everything else uses the agreed defaults: lockout after 5 wrong passwords for 15 minutes, a step-up valid for 10 minutes, one-time terminal codes valid for 24 hours, 60 minutes' notice for fee and layout changes, a 36-hour offline window, a clock warning beyond 5 minutes and a payment stop beyond 30 minutes.

> **Warning:** Never put `SEED_ADMIN_*` or `BOOTSTRAP_ADMIN_PASSWORD` in `deploy/.env`, and never run `docker compose down -v` on a server: `-v` deletes the database.

Check it

From `/opt/tppa-pos/app/deploy`:

Check	Command	Expected
State and health	`docker compose ps`	Every service "healthy"; `migrate` "Exited (0)"
The API and database	`curl -s https://<console address>/api/health`	`{"status":"ok","database":"ok"}`

Check	Command	Expected
HTTPS	Open `http://<console address>`	Redirects to `https://` and shows the sign-in page
Logs	`docker compose logs -f api` (or `web`, `nginx`, `postgres`)	No errors
Firewall	`sudo ufw status`	80, 443 and `tailscale0` only

The console works only through nginx with HTTPS: the sign-in cookie is sent over HTTPS only.

4. The first super admin

A new server has no accounts. The development **seed** (`db:seed`, with `SEED_ADMIN_USERNAME` and `SEED_ADMIN_PASSWORD`) is for a developer's PC only: it refuses to run with `NODE_ENV=production` ("The dev seed must not run in production").

On a server, use the **bootstrap command**:

```
cd /opt/tppa-pos/app/deploy
docker compose run --rm --no-deps api node dist/admin-bootstrap.js --username <username> --name "<Full Name>"
```

- It asks for the password twice and does not show it (it is never on the command line, so not in the shell history). At least 12 characters; use a password manager.
- Username: 4 to 30 letters, digits, `.` or `_`.
- It creates one super admin, and only if **no super admin exists yet**, active or not, so it can never be used later as a back door. Otherwise: "A super admin already exists." and nothing changes.
- Success: `Created super admin "<username>". Sign in to the web console now ...`. The audit log records it.

Then, in the console:

1. Sign in with the new account. The console asks to **set up two-step sign-in** at once: confirm the password, scan the QR code with an authenticator app (Google Authenticator, Microsoft Authenticator or similar), save the 10 recovery codes somewhere safe and apart from the phone, tick the box and type the code from the app.



1. Scan with the authenticator app
2. Can't scan? Type this key
3. Recovery codes: write them down
4. Confirm you saved them
5. The 6-digit code the app shows
6. Turn on two-step sign-in

Scan the QR code, save the recovery codes, enter the first code.

2. **Create a second super admin** for another trusted person (Users, **New user**, role Super admin), so that a lost phone never locks head office out. Only super admins create or change administrators.
3. Set up head office in this order: branches; admins and the other users; the utilities with their three codes (utility code, distribution code, client code); enable each utility at each branch (Branches, the branch, Utilities at this branch); the fee versions; the receipt layouts (the counter form, and the thermal roll for kiosks); then the terminals and their OR series. The Overview's **Needs your attention** list shows what is still missing.

The page that sets up two-step sign-in says "Required for administrators". On a server set up with the stack, it is required for super admins and admins.

5. Installing the teller app on a counter PC

What the counter needs

Item	Requirement
PC	Windows. The teller app is built and tested on Windows.
Windows account	One Windows account that always runs the app. The app's keys are sealed by Windows for that account on that PC (see "Data on the terminal").
Clock	Date and time set automatically. Beyond 5 minutes off the app warns; beyond 30 minutes it stops payments.
Network	HTTPS to the console address. The app works offline, but it must reach the server to register, to sync, for kiosk credit refunds, and at least once every 36 hours.
Receipt printer	An **Epson LX-310** dot-matrix printer, installed in Windows with **Epson's LX-310 driver** (with its status monitor). The app sends the receipts to the Windows print spooler as raw ESC/P, at 15 characters per inch, on the pre-printed 8.5" × 5.5" forms.

Item	Requirement
Forms	The pre-printed receipt forms, loaded in the LX-310.

Printer drivers that cause trouble:

- A printer that refuses raw data (for example "Microsoft Print to PDF") cannot print receipts: the app says "The chosen printer cannot print receipts (it is not the LX-310, or has the wrong driver)."
- A "Generic / Text Only" driver prints, but never reports out of paper or offline. Install Epson's driver with its status monitor.

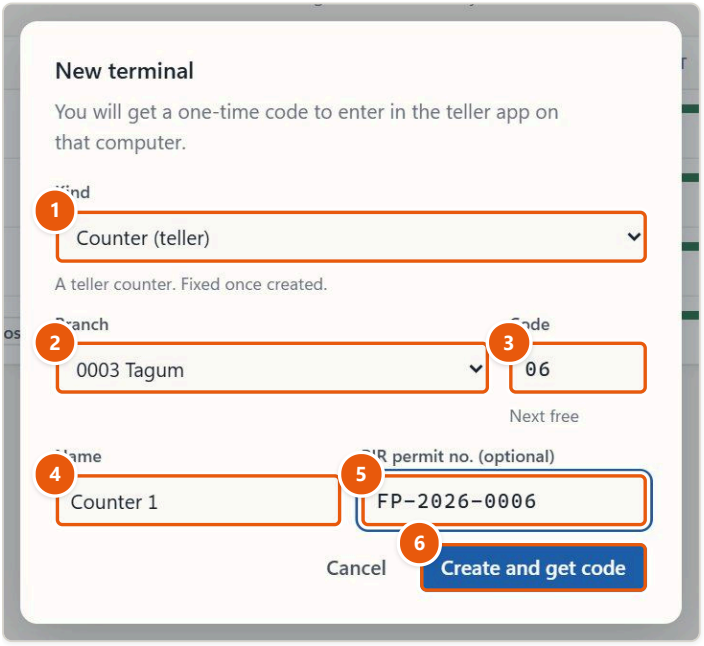
Install the app

The repository has no Windows installer for the teller app yet. Install the build the development team provides, from a signed release only, never from source code. The app keeps its data in `%APPDATA%\TPPA Teller` for the Windows account that runs it.

Create the terminal in the console

An admin does this, in the console:

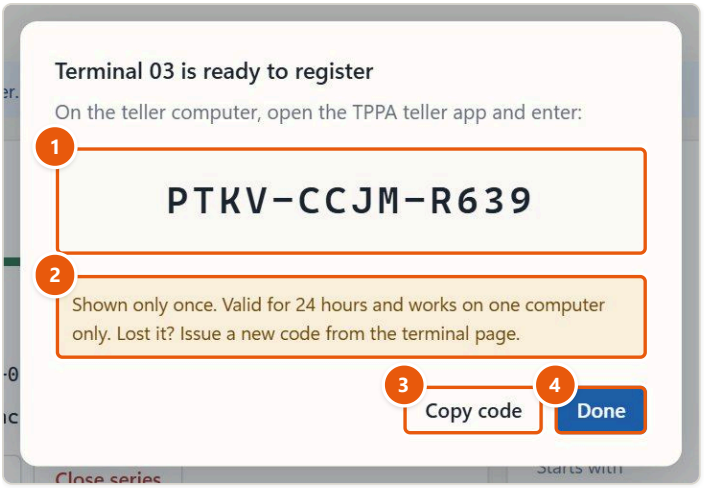
1. **Terminals**, **New terminal**.
2. **Kind:** "Counter (teller)". It is fixed once created.
3. **Branch**, **Code** (the next free one is suggested), **Name** (for example "Counter 3") and **BIR permit no.** (optional now, needed before go-live).
4. **Create and get code**. The console asks for your password again (step-up).



1 Counter or kiosk: fixed once created
2 Branch
3 Code: the next free one
4 Name
5 BIR permit no.
6 Create and get code

A new counter terminal (left with Cancel here).

5. The console shows the **one-time code**: "Shown only once. Valid for 24 hours and works on one computer only. Lost it? Issue a new code from the terminal page." Copy it and give it to the person at the PC by a safe channel.



1 The one-time code
2 Shown once, 24 hours, one computer
3 Copy code
4 Done

The code to type in the teller app on that computer.

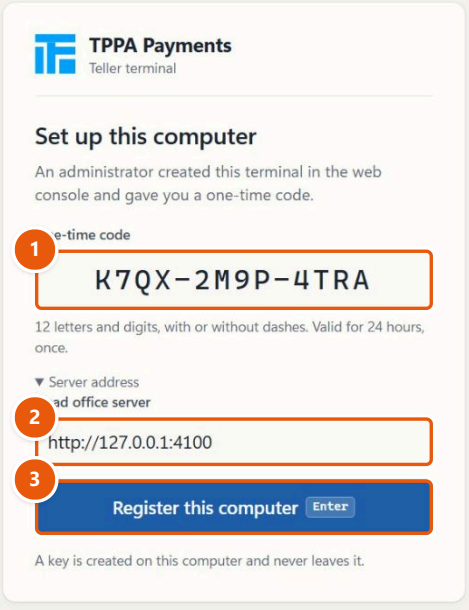
6. On the terminal's page, **Open OR series** if the terminal has none: use the range on the BIR permit for this terminal. Ranges never overlap and numbers are never reused.

A code that expired or was lost: open the terminal (status "Waiting for setup") and press **Issue new code**. The old code stops working.

Register the PC

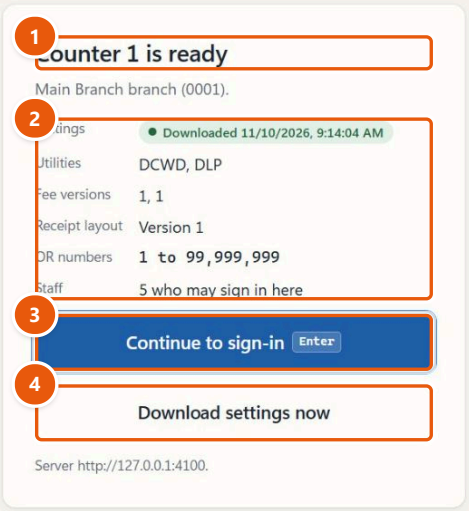
At the counter PC:

1. Start the teller app. It opens on **Set up this computer**.
2. Type the **one-time code** (1). Dashes are optional.
3. Open **Server address** (2) and check it is the console address, `https://<console address>`. The address must start with `https://`.
4. Press **Register this computer** (`Enter`). "A key is created on this computer and never leaves it."



1 Type the one-time code from the administrator
2 Server address: change only if IT says so
3 Register this computer (Enter)

5. The app shows "Counter 1 is ready" with what it downloaded: the utilities, the fee versions, the receipt layout, the OR numbers and how many staff may sign in here. If the settings did not arrive, it says "Connect to the network and download the settings before taking payments.": fix the network and press **Download settings now**.



1. The terminal and its branch
2. What was downloaded from head office
3. Continue to sign-in (Enter)
4. Download the settings again

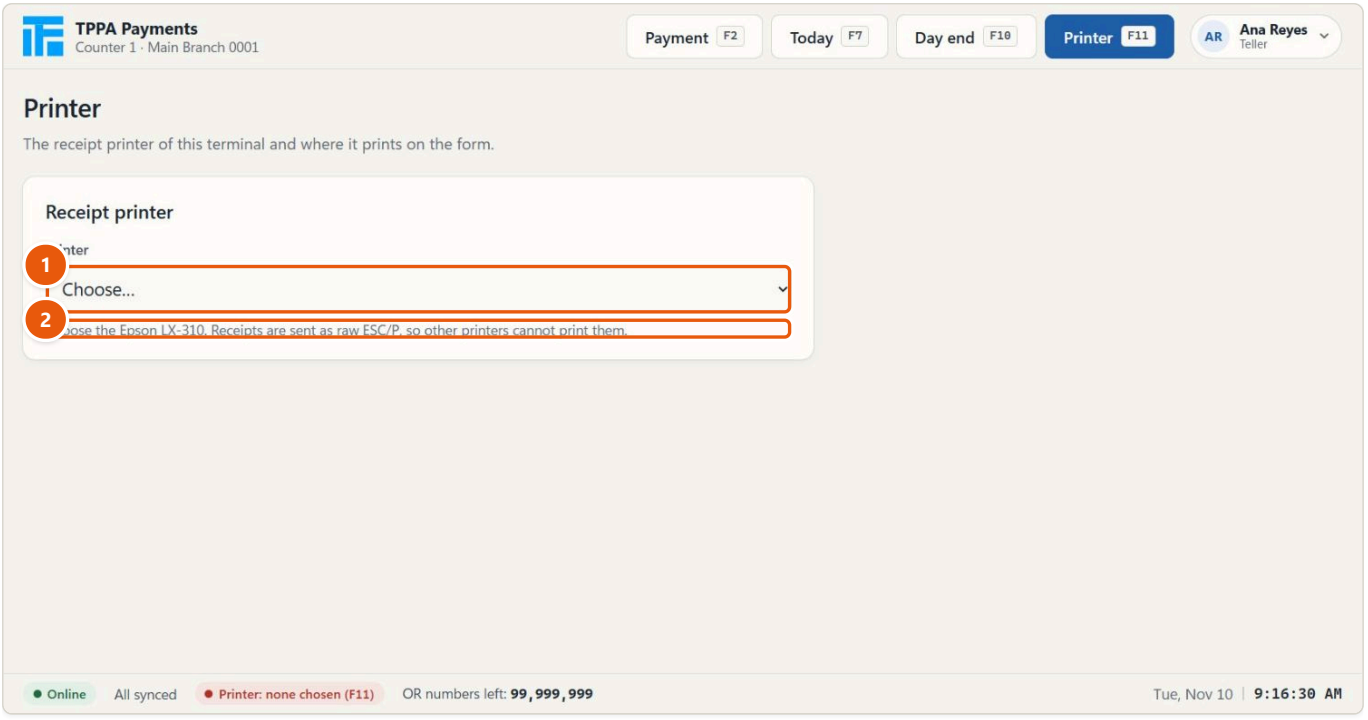
6. Press **Continue to sign-in** (`Enter`).

If the code is refused, see "The set-up code is refused" in the Troubleshooting guide. A code made for a kiosk is refused by the teller app, and the other way round.

Choose and calibrate the printer

A teller or the branch manager signs in, then:

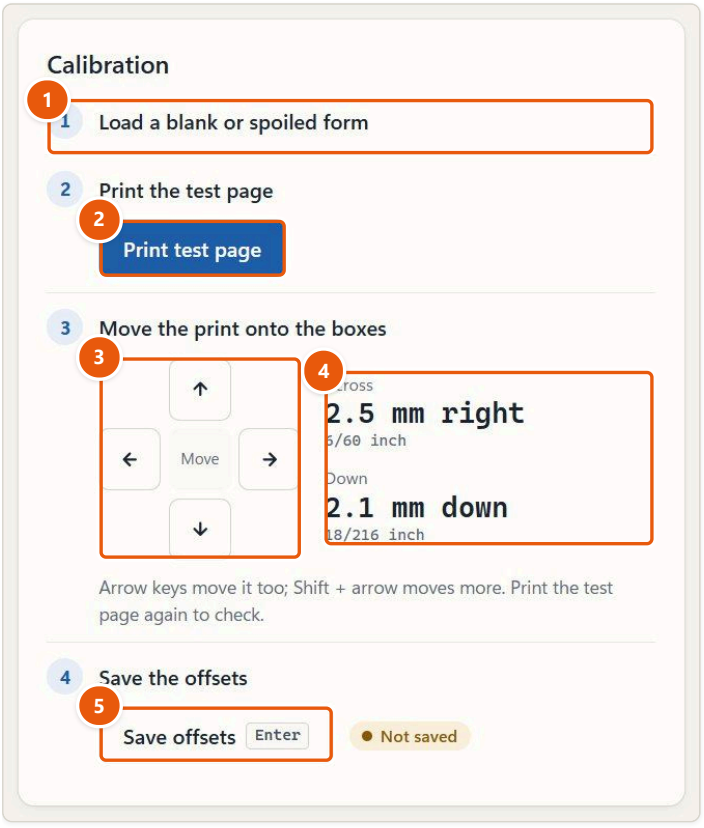
1. Press `F11` (**Printer**). Until a printer is chosen, the payment screen says "No receipt printer is chosen for this terminal. Open Printer (F11) and choose the LX-310."
2. In **Printer**, choose the Epson LX-310.



1 Choose the receipt printer

2 Only the Epson LX-310 prints receipts

3. **Calibration:** load a blank or spoiled form, press **Print test page**, move the print onto the boxes (or with the arrow keys; Shift + arrow moves more), print the test page again to check, then **Save offsets** (`Enter`). Each counter keeps its own offsets.



1 Load a blank or spoiled form

2 Print the test page

3 Move the print (or arrow keys)

4 How far it moved

5 Save the offsets (Enter)

4. Print a sample receipt on a spoiled form to check every field lands in its box.

The counter can now take payments once head office has, for its branch, a utility enabled with a fee in force, a receipt layout in force and an OR series. If anything is missing, the payment screen says "Payments cannot be taken right now." with the reason.

6. Installing a kiosk

The hardware and the PC

The kiosk design calls for a note validator for Philippine pesos with escrow and a lockable cash box, a ccTalk coin validator, an 80 mm thermal printer with an auto-cutter and a paper-near-end sensor (ESC/POS), a 2D barcode scanner, a Windows 11 IoT Enterprise PC with a TPM, and a UPS.

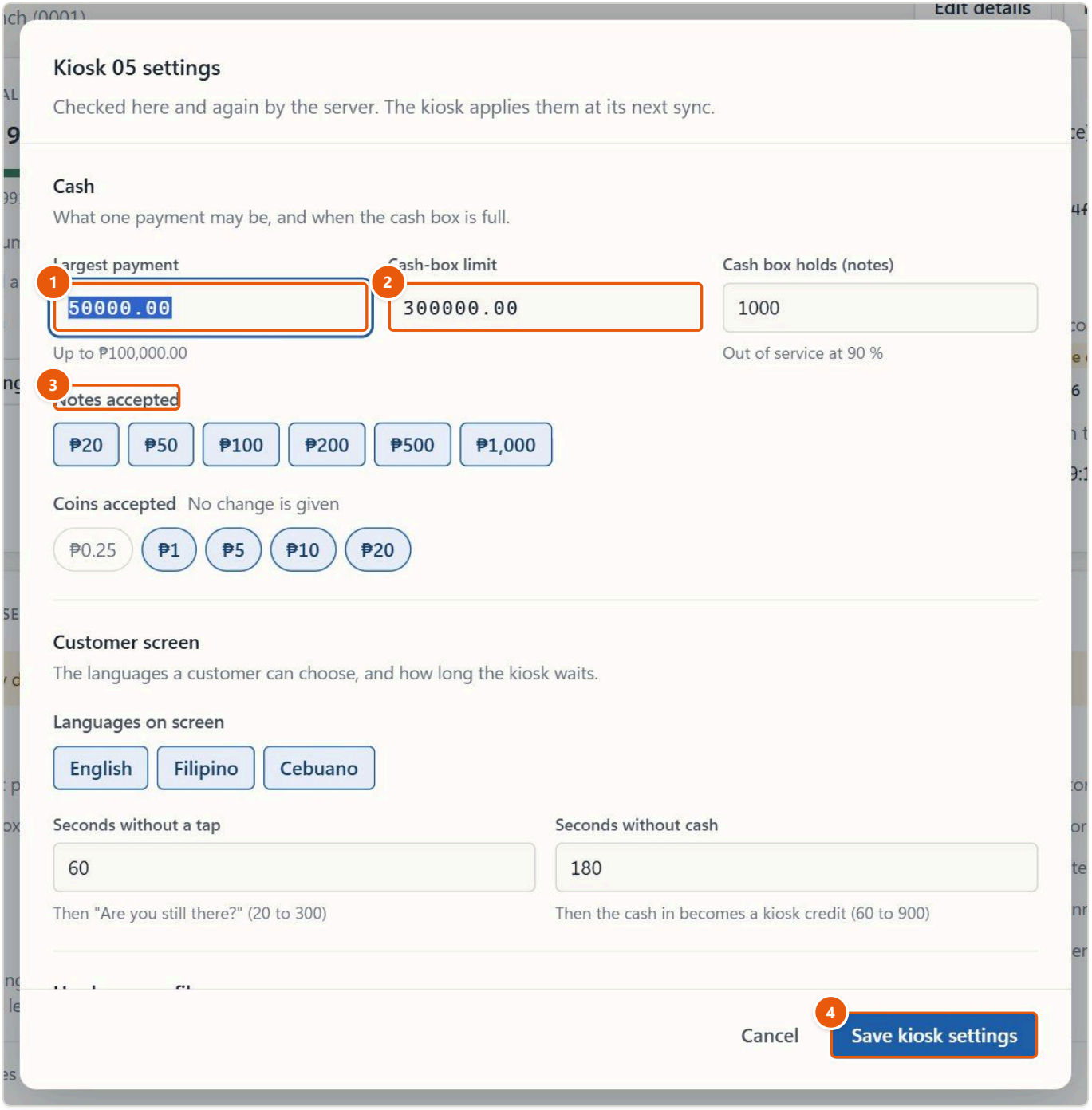
It also calls for a locked-down PC, because the kiosk is an unattended public machine:

- **Windows 11 IoT Enterprise** with the kiosk app as the only shell (Shell Launcher or Assigned Access);
- **BitLocker** with the TPM, and a firmware password;
- no keyboard outside the cabinet, and the USB ports inside the locked cabinet.

The installed (packaged) kiosk app always opens locked: kiosk mode, full screen, no menu and no keyboard shortcut to leave it. Its data is in `%APPDATA%\TPPA Kiosk`, apart from a teller's.

The hardware profile and the drivers

Each kiosk has a **hardware profile** in the console: a driver for each device (Terminals, the kiosk, **Edit kiosk settings**, Hardware profile).

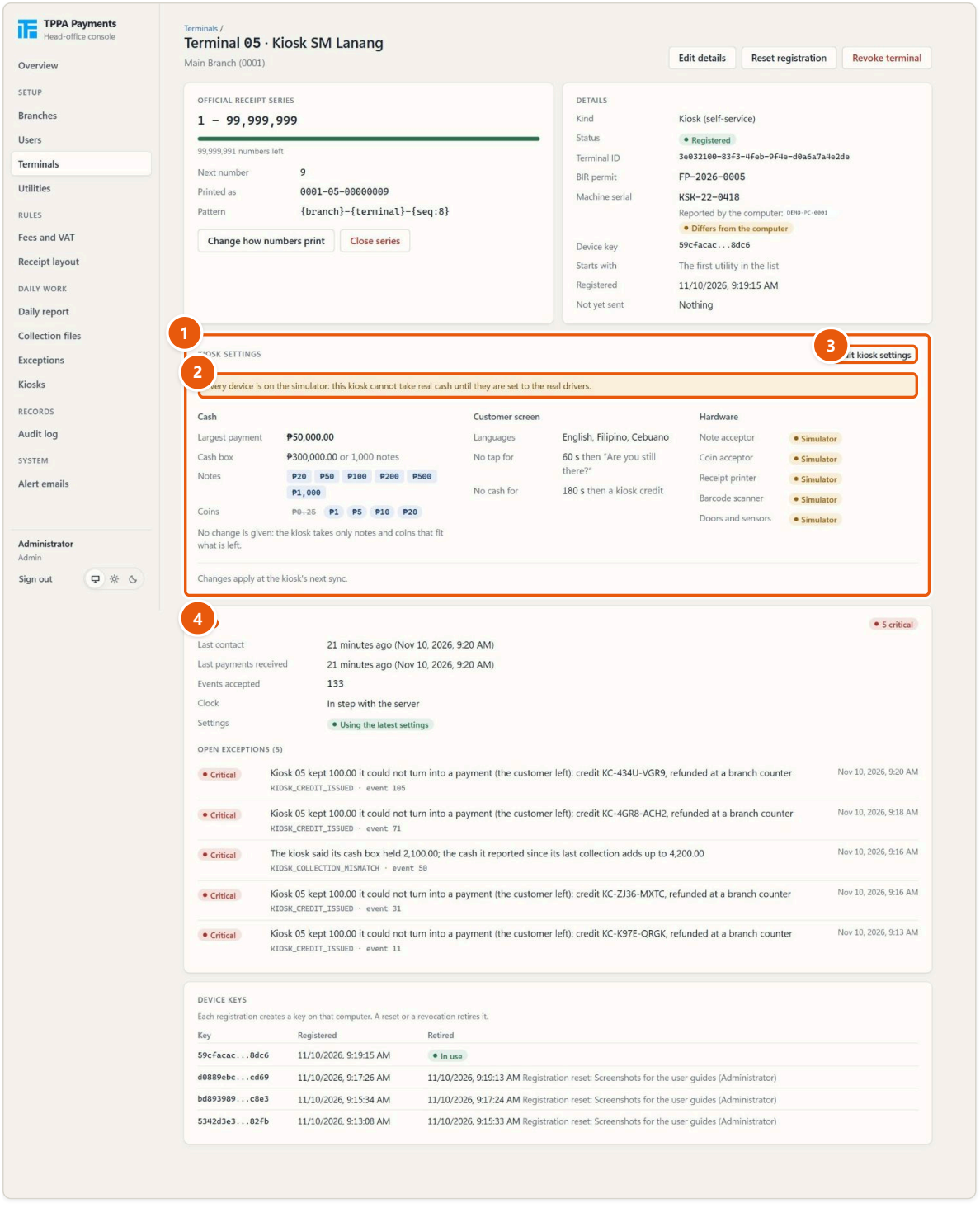


1. Largest payment
2. Cash-box limit
3. Notes and coins accepted
4. Save kiosk settings

Device | Drivers in the console | In this version of the kiosk app |

| ---------- | ---------- | ---------- | | Note acceptor | Simulator, SSP (Innovative Technology), ID-003 (JCM) | Simulator only | | Coin acceptor | Simulator, ccTalk | Simulator only | | Receipt printer | Simulator, ESC/POS thermal printer (a Windows printer) | Simulator; ESC/POS through the Windows print spooler, with the printer's Windows name | | Barcode scanner | Simulator, keyboard mode, serial | Simulator; keyboard mode | | Doors and sensors | Simulator, USB I/O board | Simulator only |

A device set to a driver this version does not have keeps the kiosk out of service, with the reason in staff mode: "...: the "..." driver is not in this version of the kiosk app yet. Choose the simulator in the console, or update the kiosk app." The real drivers arrive with the chosen hardware. While every device is on the simulator, the console warns: "Every device is on the simulator: this kiosk cannot take real cash until they are set to the real drivers."

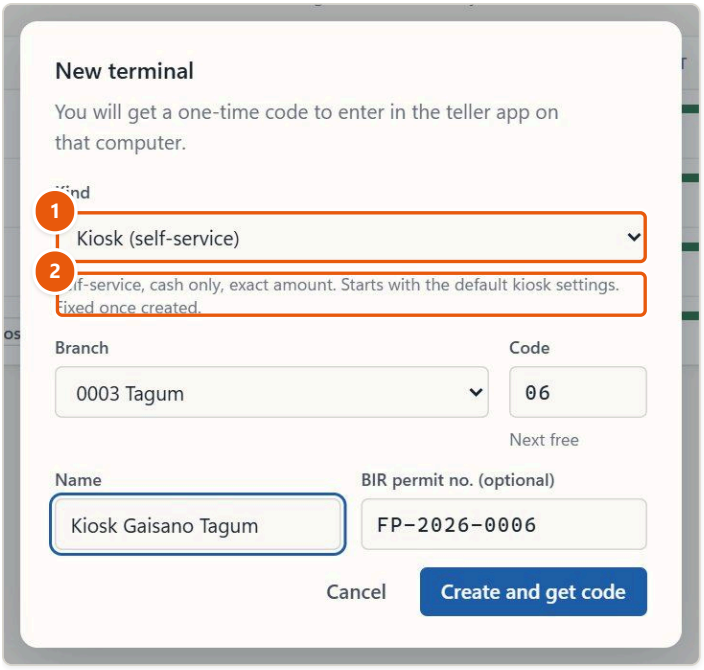


1. Kiosk settings
2. Simulated devices take no real cash
3. Edit kiosk settings
4. Sync, as for a counter

The kiosk's terminal page, with its settings and the simulator warning.

Create and register the kiosk

1. In the console: **Terminals**, **New terminal**, **Kind:** "Kiosk (self-service)". It starts with the default kiosk settings. **Create and get code**, then open its OR series.

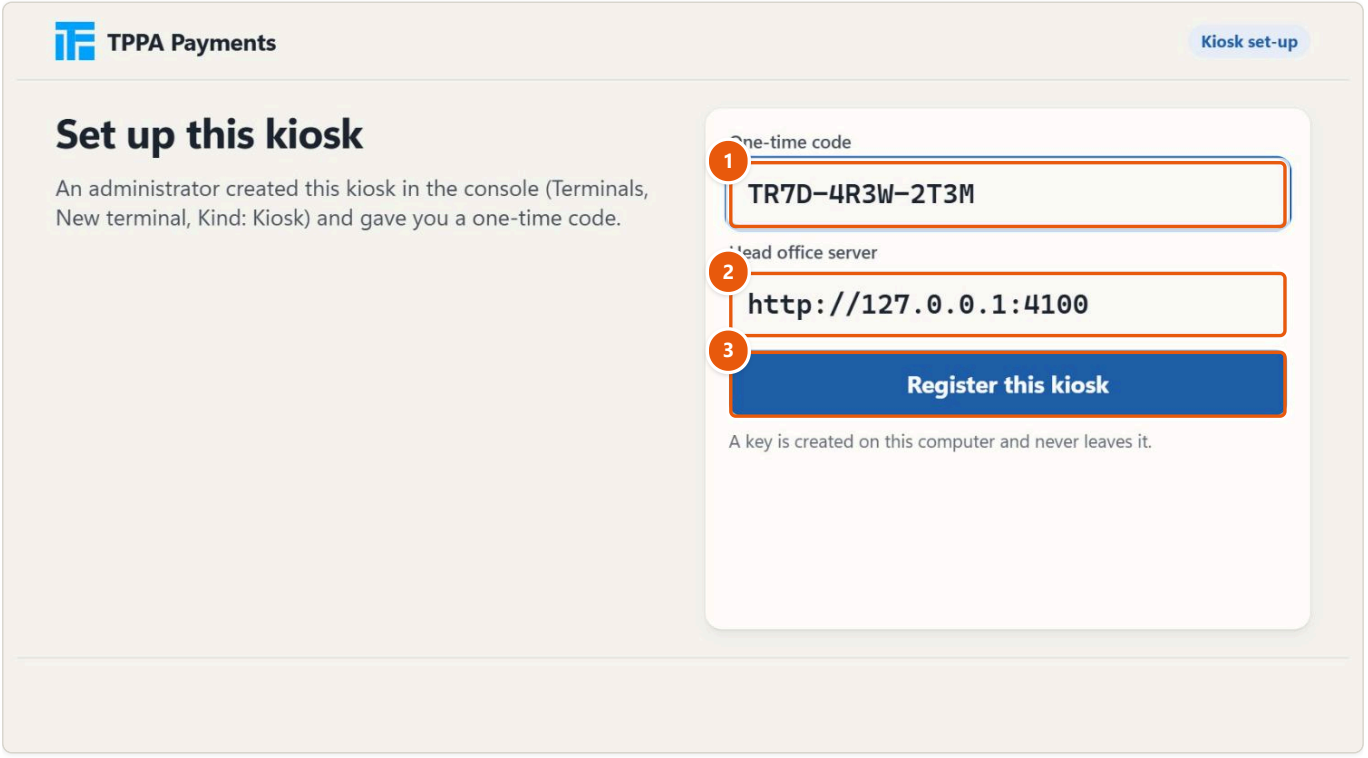


1 Kiosk (self-service)

2 Starts with the default kiosk settings

A new kiosk (left with Cancel here).

2. Set the kiosk's settings (largest payment, cash-box limit, notes and coins, languages, the time-outs) and its hardware profile. Changes apply at the kiosk's next sync.
3. At the kiosk, the app opens on **Set up this kiosk**. Type the one-time code (1), check **Head office server** (2) and press **Register this kiosk** (3).

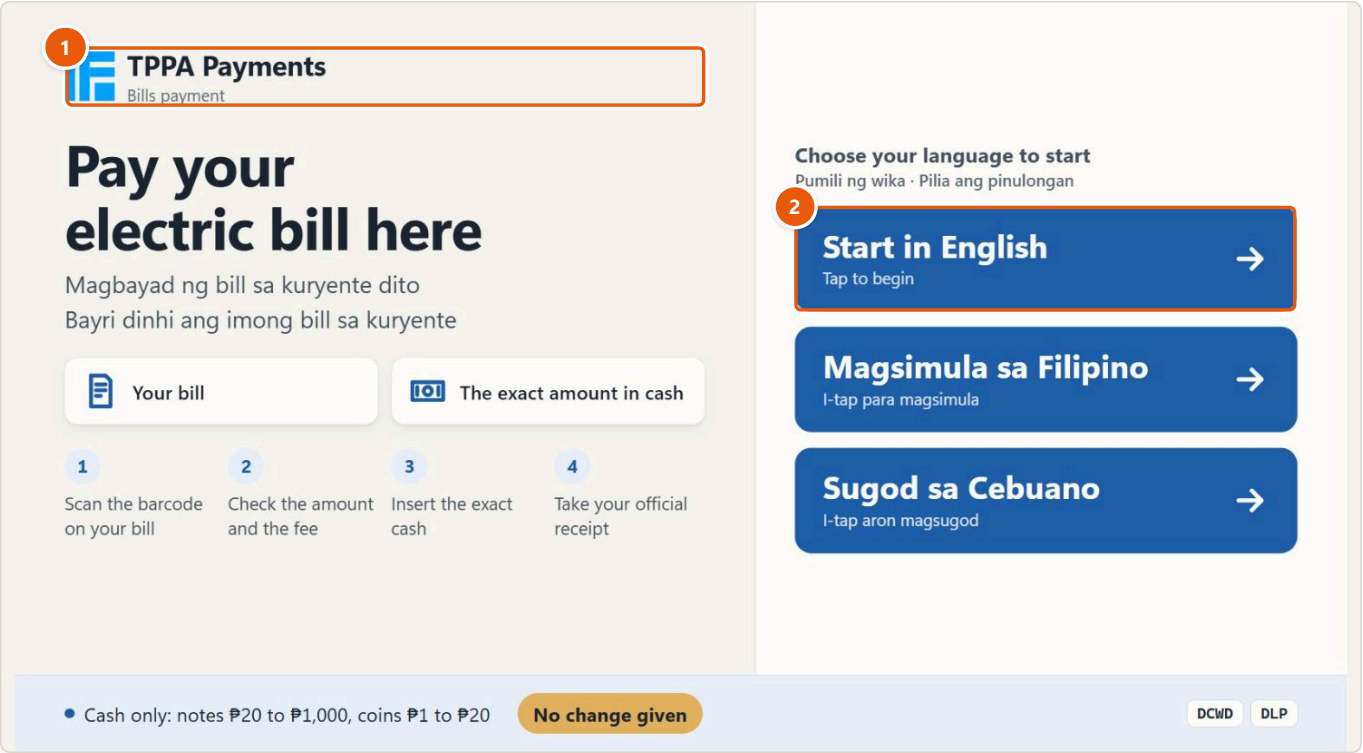


1 The one-time code from the console

2 Head office server (filled in)

3 Register this kiosk

4. Once the settings are in, the kiosk shows the start screen, ready for customers. It needs a utility with a fee in force at its branch, a receipt layout and an OR series; otherwise it shows "Out of service", with the reason in staff mode.



1 Registered: the kiosk is ready for customers

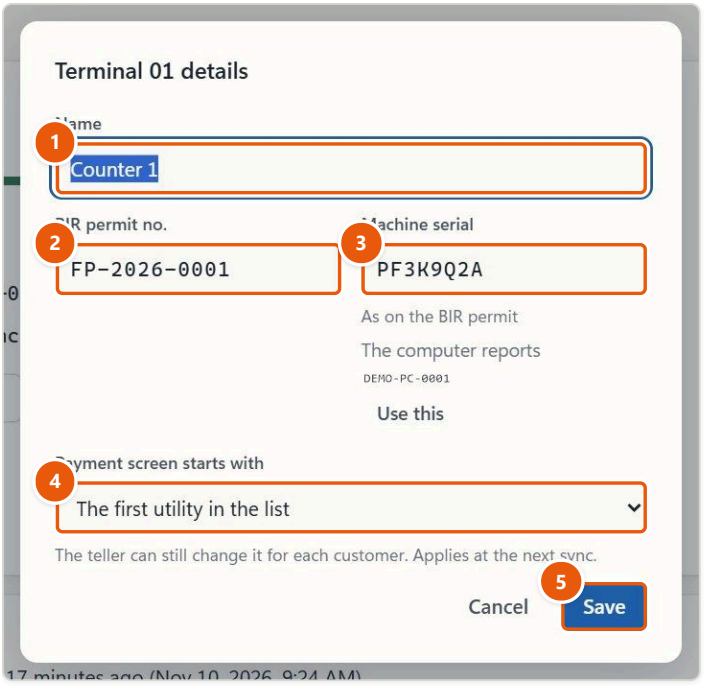
2 A customer starts here

A code made for a counter is refused: "This code is for a teller counter, not a kiosk. Enter it in the TPPA teller app, or ask the administrator for a kiosk code (Kind: Kiosk)."

7. Machine serial and BIR permit

The BIR permit of a terminal follows its machine. In the console, each terminal has:

- **BIR permit no.**: as on the permit (Terminals, the terminal, **Edit details**).
- **Machine serial**: "As on the BIR permit". The app reports the computer's own serial (from the BIOS) at each sync, and the console shows it beside the typed one ("The computer reports ...", with **Use this**). When they differ, the terminal page shows "Differs from the computer".



1 Name
2 BIR permit no.
3 Machine serial, as on the permit
4 The utility the payment screen starts with
5 Save

A terminal's details (left with Cancel here).

If a terminal starts reporting another serial, head office gets a critical **Machine serial changed** exception and an alert email: "Its BIR permit follows its machine: check whether it moved to other hardware." Other hardware is a new terminal (see the next section).

8. Resetting, replacing and setting up again

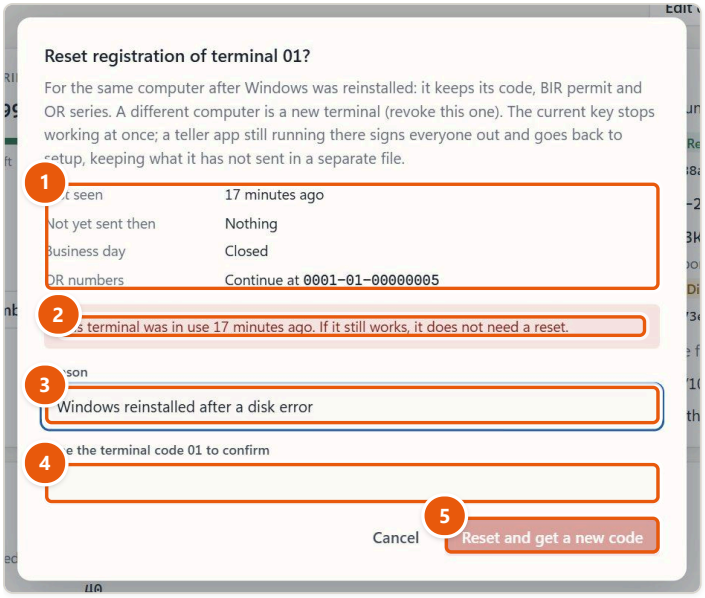
Which action to use

Situation	Action in the console
Same computer, Windows reinstalled (or the app's data lost)	**Reset registration**: the same terminal, code, BIR permit and OR series; a new one-time code; the computer makes a new key.
A different computer (hardware broken or swapped)	**Revoke terminal** and create a new terminal. The BIR permit follows the machine serial, so new hardware is a new terminal.
Lost or stolen	**Revoke terminal**.
A new terminal whose code expired before it registered	**Issue new code**.

Revoke cannot be undone: "The terminal stops working at its next sync and its OR series closes. Use it for lost, stolen or replaced computers. Same computer after a Windows reinstall? Use Reset registration instead."

Reset registration

On the terminal's page, **Reset registration**. The dialog shows what to check first: when the terminal was last seen, what it had not sent, whether its business day is open, and where its OR numbers continue. "This terminal was in use 17 minutes ago. If it still works, it does not need a reset." Give a reason; when there is a risk (seen recently, unsent records, an open day, a kiosk), also type the terminal code to confirm. **Reset and get a new code**.



1. Last seen, not yet sent, business day, OR numbers
2. What to check first
3. Reason
4. Type the terminal code to confirm
5. Reset and get a new code

Reset registration (left with Cancel here: Counter 1 keeps working).

What happens:

- The current key stops working at once.
- **A teller app still running there** signs everyone out and shows "This terminal was reset". It keeps what it had not sent in a separate file. If records were not sent, the screen names them (with the OR numbers) and asks the teller to keep the paper receipts, call head office and tick "I have told head office about the N records". Then **Set up this computer again** restarts the app on "Set up this computer", where the new code is typed.
- **A kiosk** shows customers "Out of service". In staff mode, the cash is collected first (the kiosk refuses to be set up again while its box holds cash), then **Set up again** and the new code.
- The new set-up continues from head office: the same OR series, from the next number head office has, and the open day's figures. If the old set-up had printed receipts head office never received, the teller app warns that those OR numbers may be printed again, and asks to confirm that head office was told.
- An old set-up that stays offline cannot be told. It stops at the end of the 36-hour offline window, like any terminal that cannot sync.

A terminal the server does not recognise

When the server answers but does not know the computer for several minutes, the app shows "The server does not recognise this computer" (a kiosk: "Out of service" for customers, the details in staff mode). Payments stop there, nothing is deleted, and nobody can sign in. It comes back by itself as soon as the server knows the computer again.

1. **Read the screen.** It says which case it is: the server has **no such terminal** (the wrong server, or its database was replaced), or it **does not know this computer's key**. It shows the server address, the terminal code and ID, the device key (short form), the last successful sync, and any records not sent with their OR numbers.
2. **Check the address.** Is it the production server, not a test one? A wrong address is corrected on the same screen (**Server address**), together with a code for the right server.
3. **In the console, Terminals:** find the terminal by its code and compare its **Device keys** with the key on the screen.
   - Not there (database replaced, or restored from before it existed): create the terminal (Kind: Counter or Kiosk, as before) and note its code.
   - There, waiting for setup: **Issue new code**.
   - There, registered with another key: **Reset registration** (it gives a new code).
   - There with this very key: the server should know it. Do not set it up again; call the development team.

4. **Unsent records:** keep the paper receipts and tell head office; the screen asks to confirm it.
5. **Kiosk:** open the service door (staff mode) and collect the cash first; the kiosk refuses a new code while the cash box holds cash.
6. **Enter the code** under "New one-time code" and press **Set up with this code**. The server checks it with a fresh key first: a wrong or expired code changes nothing. When it is accepted, the old files move to a set-aside folder and the app restarts as the new registration. A teller app then shows "... is ready"; sign in as usual.

9. Data on the terminal

- **Encrypted:** each terminal keeps its own database (settings, staff sign-ins for offline use, every payment and event) encrypted on disk. Its database key and its device key are sealed with Windows' data protection (DPAPI), tied to that Windows account on that PC: copying the files to another PC does not reveal them.
- **The device key never leaves the PC.** Every request to the server is signed with it. The console shows its short fingerprint under **Device keys**.
- **Where:** `%APPDATA%\TPPA Teller` (teller) and `%APPDATA%\TPPA Kiosk` (kiosk), for the Windows account that runs the app.
- **Set aside, never deleted:** when a terminal is set up again (after a reset or with a new code), its old database and keys move to `set-aside\<time>\` in the same data folder. They stay readable on that PC and Windows account only. Unsent payments are real cash: never delete a terminal's data folder.

> **Warning:** Do not change the Windows account that runs the app, and do not copy the data folder to another PC: the keys open only for the same account on the same computer. If you must reinstall Windows, first let the terminal sync ("All synced"), then use **Reset registration**.

10. Backups and updates as they stand

Backups (manual today)

Continuous off-site backups are not built yet. Until then, set up a **nightly database dump** as the `tppa` user and copy it off the server:

1. `mkdir -p /opt/tppa-pos/backups`.
2. Add the nightly `pg_dump` line from `docs/guides/running.md` section 2.10 to the `tppa` user's crontab. It keeps 14 days of dumps.
3. Copy the backup folder to storage at a different provider (for example with `rclone`).
4. **Test a restore** into a scratch database at least once before go-live.

Records must be kept for 10 years.

Updating the server (manual today)

Outside branch business hours, from `/opt/tppa-pos/app` (`docs/guides/running.md` section 2.9):

1. Back up first with `pg_dump`: migrations only go forward.
2. `git pull --ff-only`, then `docker compose -f deploy/compose.yaml up -d --build`. The migrations run before the new API starts.
3. `docker compose -f deploy/compose.yaml ps`.

If something goes wrong: `docker compose -f deploy/compose.yaml logs api migrate`. To go back, check out the previous commit and build again; if a migration must be undone, restore the dump. Teller PCs and kiosks keep working offline during the short restart and sync afterwards.

Updating the teller and kiosk apps

There is no automatic updater. Update the apps only from a signed release the development team provides, never from source code. If a terminal says "The server sent settings this version of the app does not understand. Update the app.", it is older than the server.

11. Security practices

- **Keys never leave the PC.** Each terminal signs every request with its own device key, sealed by Windows. A reset or a revocation retires the key; a used key never registers again.
- **TLS everywhere.** The console works only over HTTPS through nginx; HTTP redirects to HTTPS. A terminal's server address must start with `https://`.
- **Server access.** Only ports 80 and 443 are public; SSH only over Tailscale; the database, the API and the console have no published port. `deploy/.env` is mode 600, owned by `tppa`, and each server has its own secrets.
- **Accounts.** Everyone has their own account. Five wrong passwords lock it for 15 minutes; the server also limits sign-ins per network address. Two-step sign-in is required for super admins and admins on the server; keep two super admins, each with stored recovery codes.
- **Step-up.** Sensitive changes in the console ask for the password again ("Confirm it is you"), valid for 10 minutes on that device.
- **Four-eyes rule.** Voids, cash differences, float corrections and kiosk credit refunds are approved on the screen by a branch manager with their own password; a kiosk box is counted by someone other than its collector.
- **Audit log.** Every sign-in and change, who made it and from where. Entries can never be edited or deleted.
- **Alert emails.** Critical problems within minutes, kiosk notices, and a daily report. Set the mail server and send a test email before go-live.
- **Development data stays in development.** Never use the development seed, the test accounts or their passwords on a server. Staging holds synthetic data only.
- **Kiosks** are unattended public machines: lock the PC down as above. Staff mode opens only while the service door is open, and asks for a staff username and password.

12. Before go-live: the checklist

Server

- ☐ `docker compose ps`: every service healthy; `/api/health` answers `"ok"`.
- ☐ Public ports: only 80 and 443 (`sudo ufw status`); SSH only over Tailscale.
- ☐ `deploy/.env` mode 600, owned by `tppa`; its own `BETTER_AUTH_SECRET` and database password.
- ☐ HTTPS works, HTTP redirects to HTTPS, and the certificate renews (check with `docker compose logs certbot`).
- ☐ The nightly backup runs, is copied off the server, and a restore has been tested.
- ☐ The mail server is set; **Send a test email** reached every recipient.

Accounts

- ☐ Two super admins, both with two-step sign-in and stored recovery codes.
- ☐ Every person has their own account, with the right role and branch.
- ☐ Nobody uses the development seed or development passwords.

Head-office set-up

- ☐ Branches, with their codes.
- ☐ Each utility with its utility code, distribution code and client code; enabled at each branch that collects for it.
- ☐ A fee version in force for each utility.
- ☐ A receipt layout in force for the counter form, and for the thermal roll if there are kiosks.
- ☐ Each terminal created, with its BIR permit no., machine serial and an open OR series (the range on its BIR permit).

Each counter PC

- ☐ Windows clock set automatically; the app runs under its one Windows account.
- ☐ The Epson LX-310 installed with Epson's driver and status monitor.
- ☐ The teller app registered ("... is ready"), the printer chosen and calibrated, a sample receipt checked on a spoiled form.
- ☐ A teller signs in online once, so that offline sign-in works for them.
- ☐ The status bar reads **Online** and **All synced**; the console shows the terminal **Registered** and "In step with the server".

Each kiosk

- ☐ The PC locked down (Windows 11 IoT Enterprise, the kiosk app as the only shell, BitLocker, firmware password, USB inside the cabinet).
- ☐ The hardware profile set to the real drivers once they are available; until then the kiosk cannot take real cash.
- ☐ Registered, in service, and seen on the console's **Kiosks** page.
- ☐ A custodian has signed in to staff mode, loaded paper and tested the acceptors.